AI COURSE – II AIML/REC

CATEGORY 2 – PYTHON PROGRAMS ONLY

1. Fully Observable vs. Partially Observable

```
class VacuumAgent:
    def __init__(self, fully_observable):
        self.fully_observable = fully_observable
        self.model = {"A": "Unknown", "B": "Unknown"}

    def act(self, location, visible_rooms):
        for room, status in visible_rooms.items():
            self.model[room] = status

        if self.fully_observable:
            if "Dirty" in visible_rooms.values():
                return "Clean a dirty room"
            return "All rooms are clean"

        if self.model[location] == "Dirty":
            return "Suck"

        other = "B" if location == "A" else "A"

        if self.model[other] == "Unknown":
            return "Move to " + other
        if self.model[other] == "Dirty":
            return "Move to " + other

        return "All known rooms are clean"


fully_agent = VacuumAgent(True)
print("Fully observable:",
      fully_agent.act("A", {"A": "Clean", "B": "Dirty"}))

partial_agent = VacuumAgent(False)
print("Partially observable:",
      partial_agent.act("A", {"A": "Clean"}))
print("Agent memory:", partial_agent.model)
```

2. Known vs. Unknown

```
class LearningAgent:
    def __init__(self, known):
        self.known = known
        self.learned_rules = {}

    def act(self, location):
        if self.known:
            return "Move to B" if location == "A" else "Goal reached"

        if location == "A":
            self.learned_rules["A"] = "Move to B"
            return self.learned_rules["A"]

        self.learned_rules["B"] = "Goal reached"
        return self.learned_rules["B"]


known_agent = LearningAgent(True)
unknown_agent = LearningAgent(False)

print("Known environment:", known_agent.act("A"))
print("Unknown environment:", unknown_agent.act("A"))
print("Learned rules:", unknown_agent.learned_rules)
```

3. Static vs. Dynamic

```
def utility_based_agent(room_status, dynamic=False):
    if dynamic:
        room_status = "Dirty"
```

```
    utilities = {
        "Clean": 10 if room_status == "Dirty" else 0,
        "Move": 5
    }

    best_action = max(utilities, key=utilities.get)

    print("Environment:",
          "Dynamic" if dynamic else "Static")
    print("Room status:", room_status)
    print("Selected action:", best_action)


utility_based_agent("Clean", dynamic=False)
utility_based_agent("Clean", dynamic=True)
```

4. Discrete vs. Continuous

```
def discrete_agent(location):
    if location == "A":
        return "Move Right"
    return "Move Left"


def continuous_agent(position, target=10.0):
    if position < target:
        return "Move Forward"
    elif position > target:
        return "Move Backward"
    return "Stop"


print("Discrete agent:", discrete_agent("A"))
print("Continuous agent:", continuous_agent(6.5))
print("At target:", continuous_agent(10.0))
```

5. Deterministic vs. Stochastic

```
import random


def deterministic_clean():
    return "Clean"


def stochastic_clean():
    if random.random() < 0.7:
        return "Clean"
    return "Dirty"


print("Deterministic result:", deterministic_clean())

print("Stochastic results:")
for i in range(5):
    print("Attempt", i + 1, ":", stochastic_clean())
```

6. Episodic vs. Sequential

```
def episodic_agent(task_status):
    if task_status == "Dirty":
        return "Suck"
    return "No action needed"


class SequentialAgent:
    def __init__(self, goal):
        self.goal = goal
        self.location = "A"

    def act(self):
        if self.location == self.goal:
            return "Goal reached"

        self.location = "B" if self.location == "A" else "A"
        return "Move to " + self.location


print("Episodic task 1:", episodic_agent("Dirty"))
print("Episodic task 2:", episodic_agent("Clean"))

agent = SequentialAgent("B")
print("Sequential step 1:", agent.act())
print("Sequential step 2:", agent.act())
```